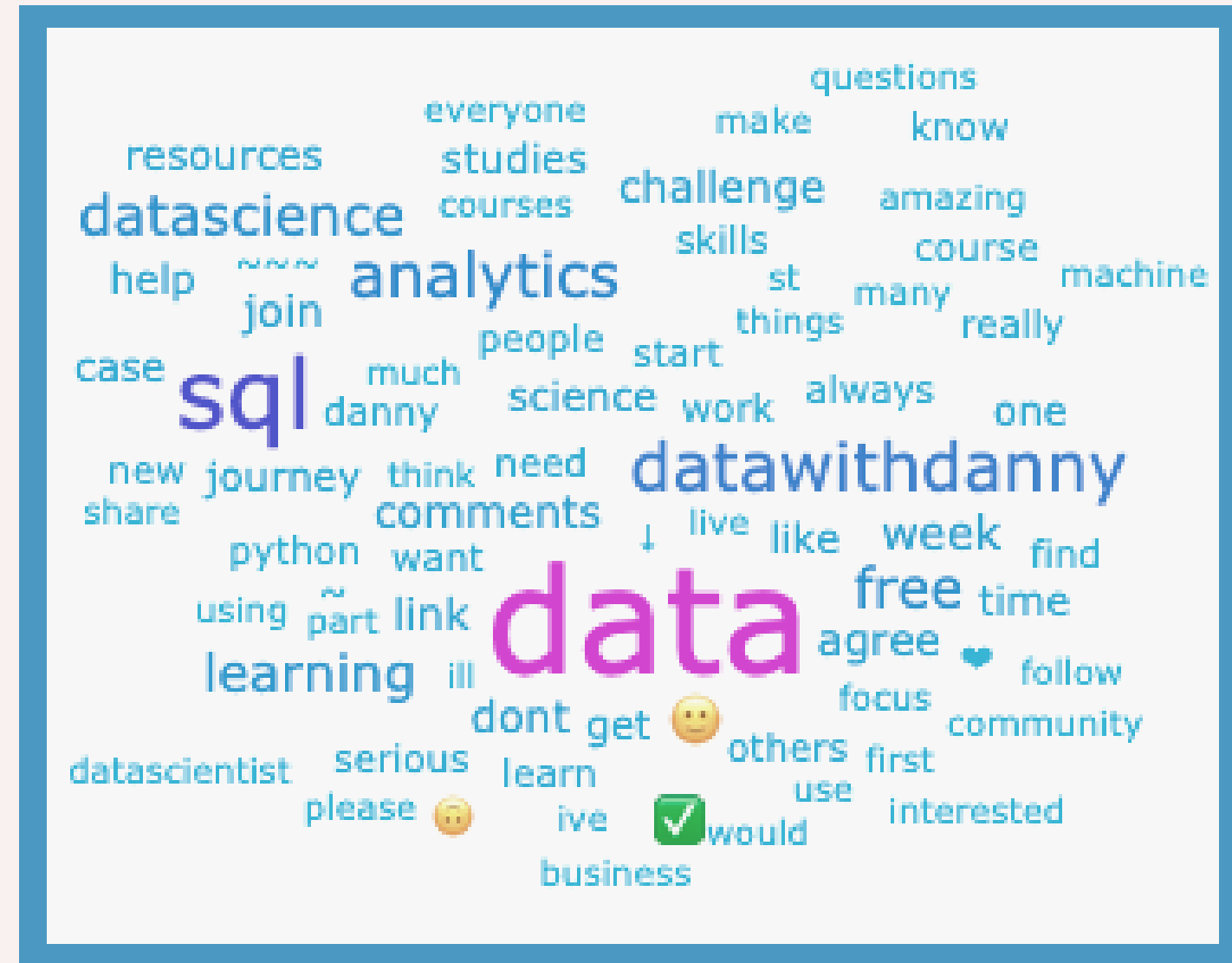
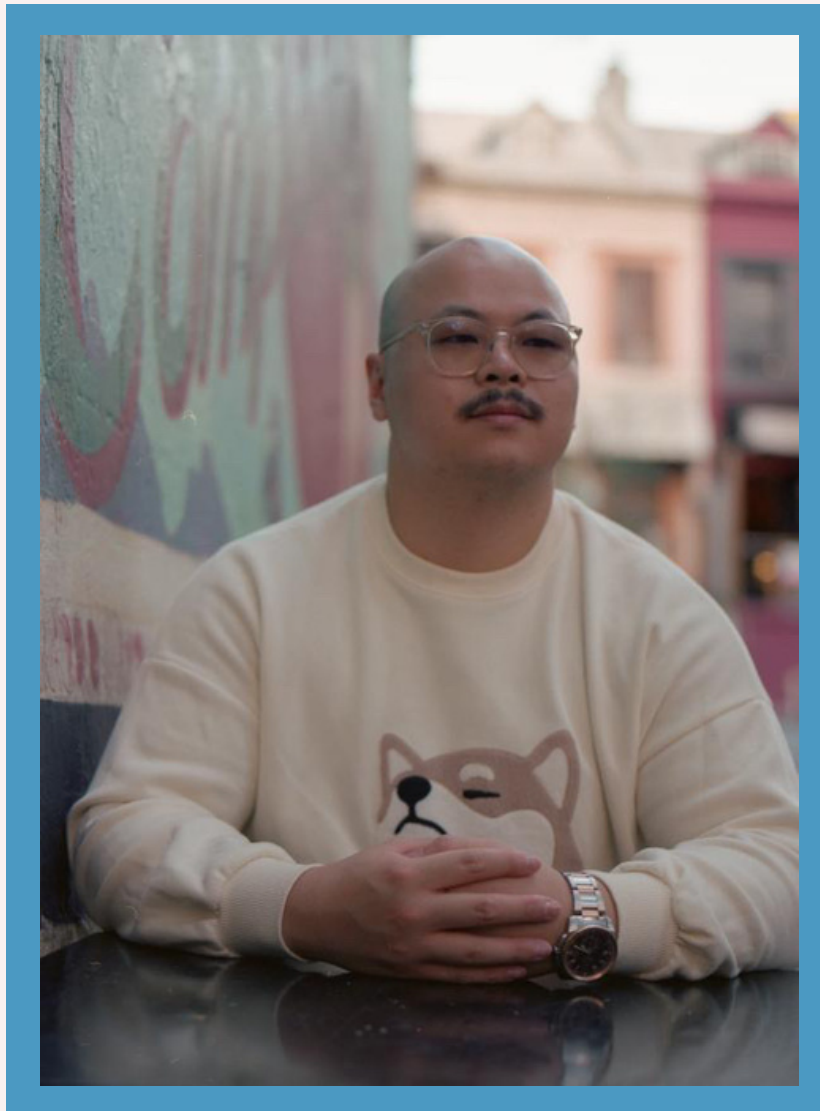




PostgreSQL for Data Analytics

Danny Ma
May 2022

About me



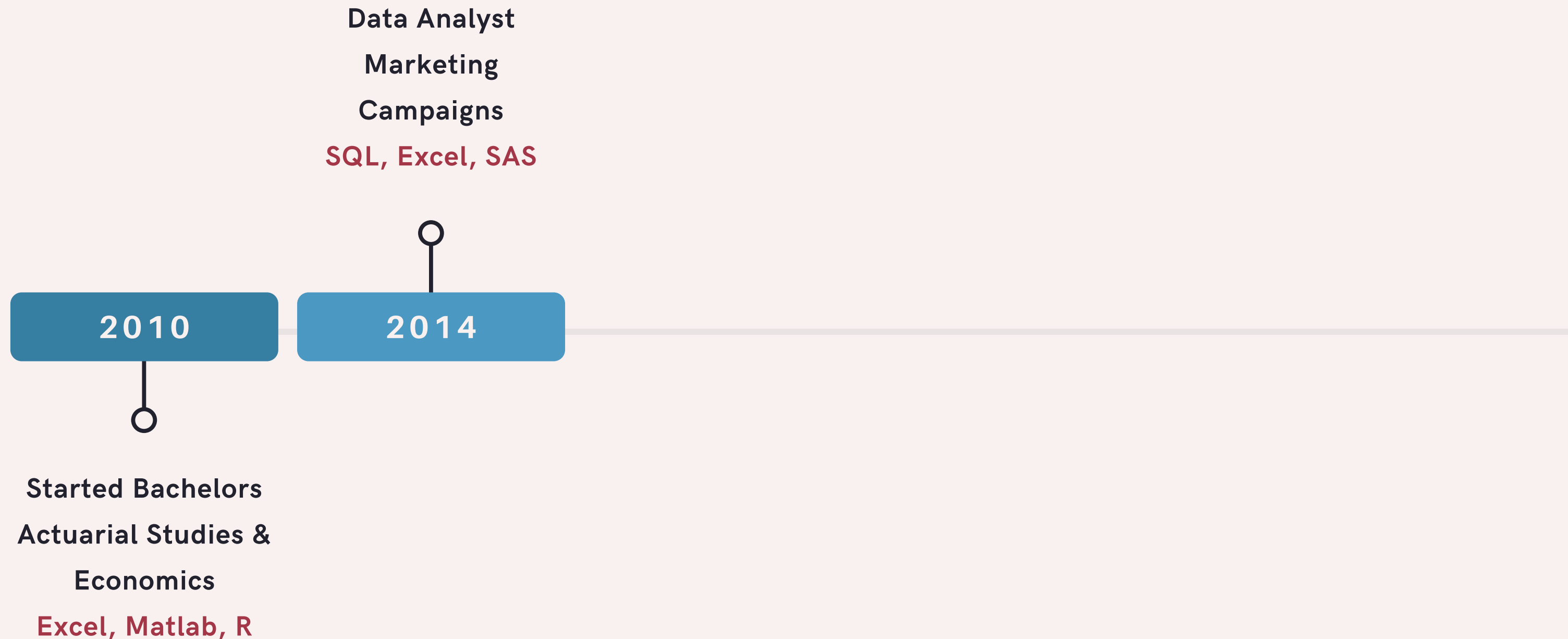
My Data Analytics Journey

My Data Analytics Journey

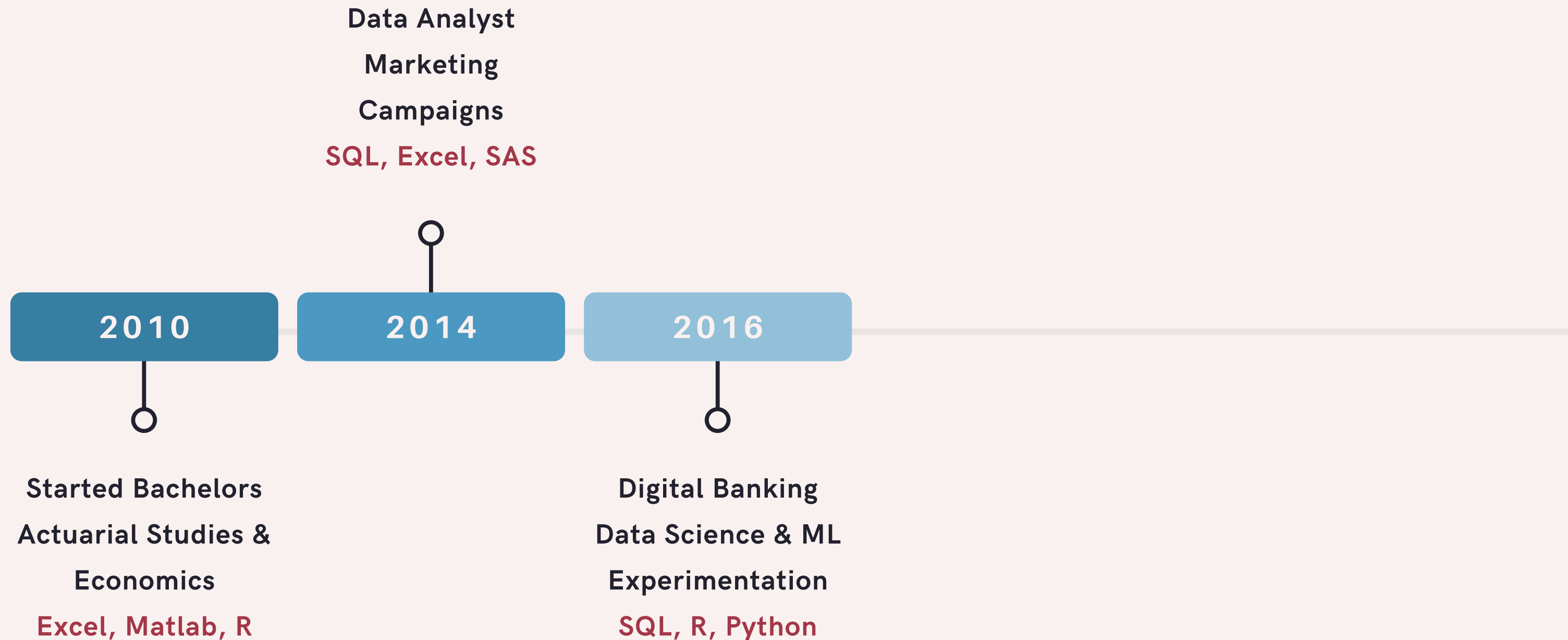
2010

Started Bachelors
Actuarial Studies &
Economics
Excel, Matlab, R

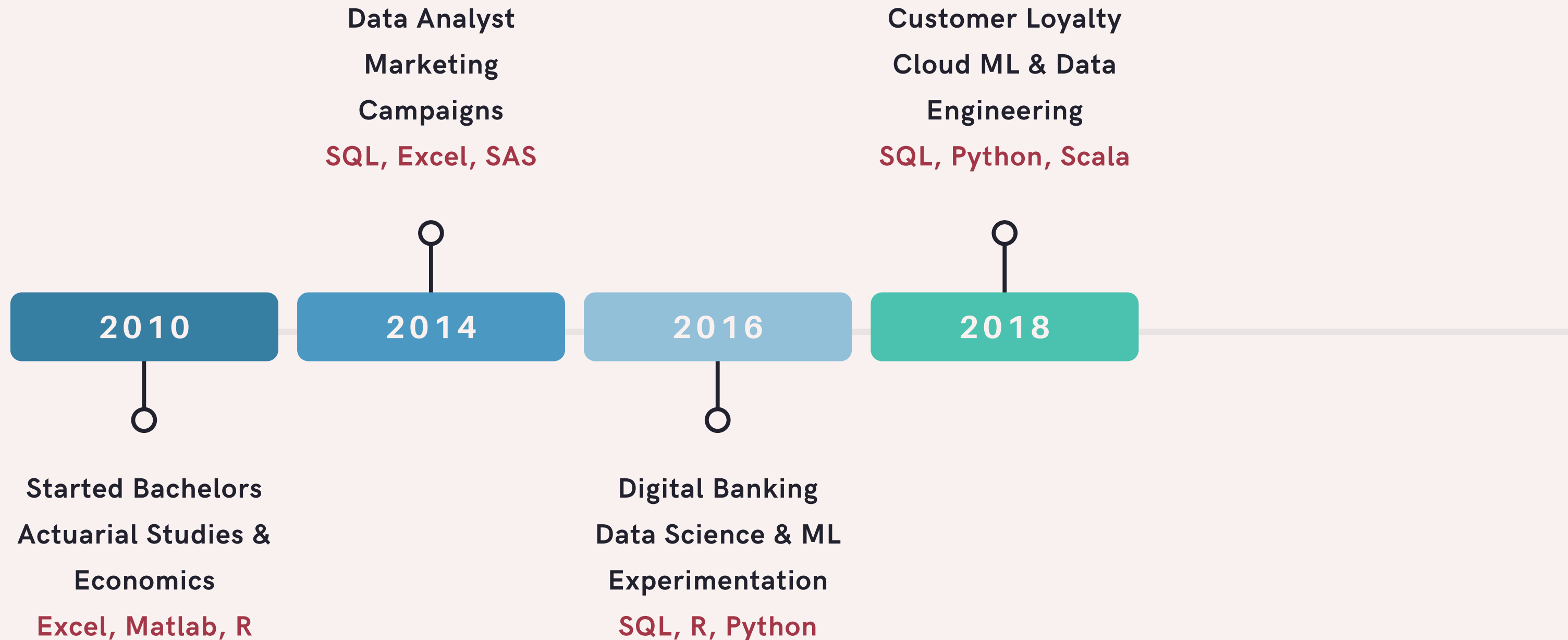
My Data Analytics Journey



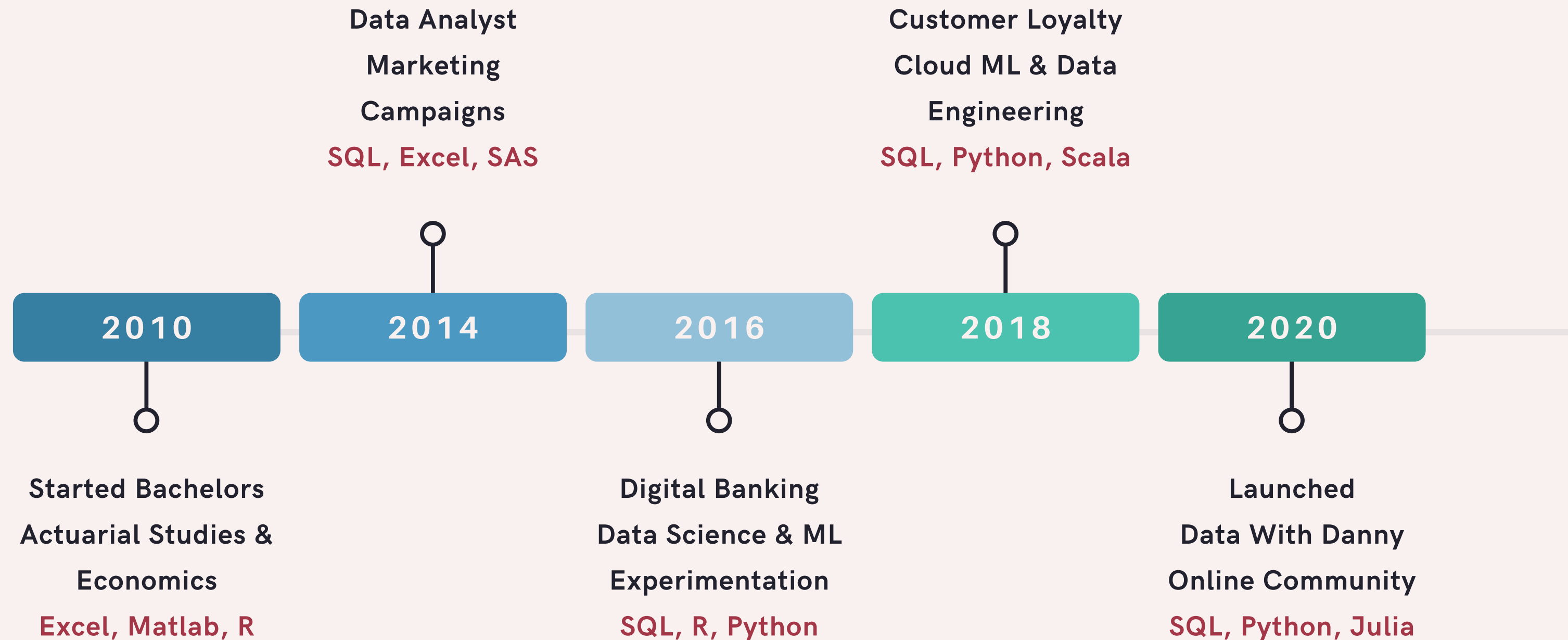
My Data Analytics Journey



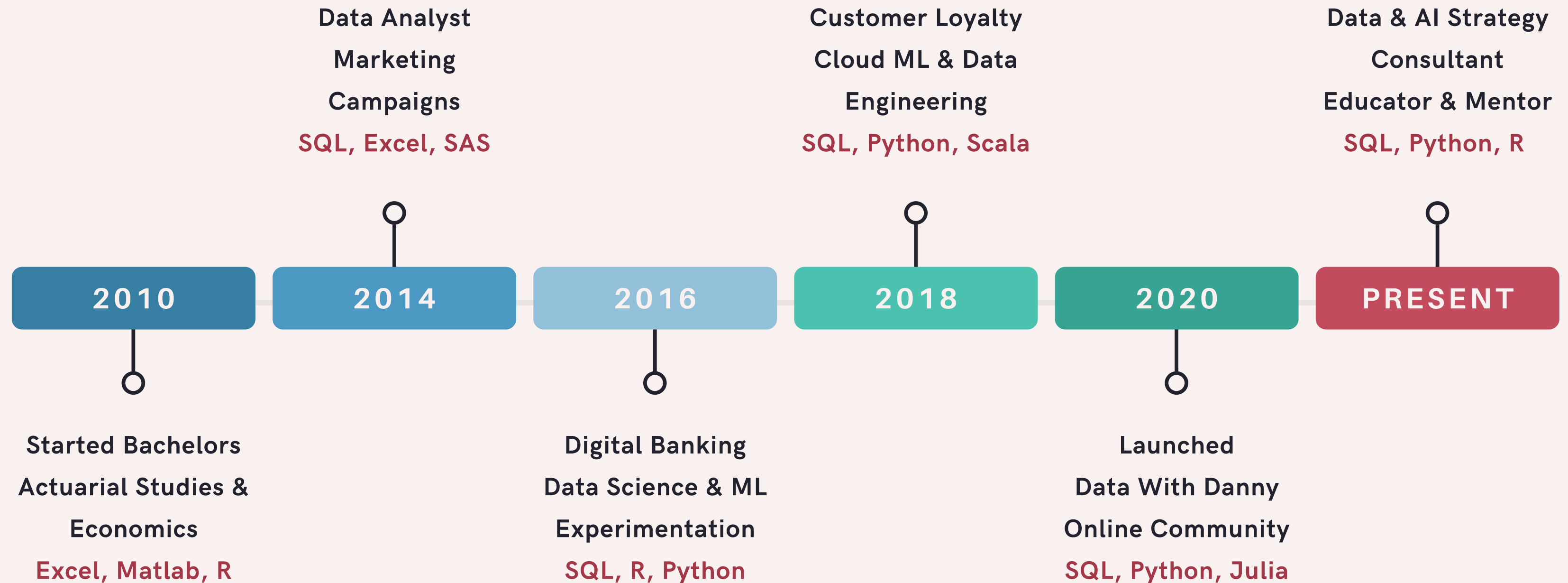
My Data Analytics Journey



My Data Analytics Journey



My Data Analytics Journey





DATA WITH DANNY

VIRTUAL DATA APPRENTICESHIP

How can I find
out more about
Data With Danny?



bit.ly/dwd-info

SQL FACTS



SQL FACTS

SQL appears in over 40%
of all data job listings
posted on Indeed in 2021

SQL FACTS

65% of data analysts
and data scientists use
SQL according to a 2020
Stack Overflow survey

SQL FACTS

SQL queries can be used
with many languages
and frameworks via
objects and APIs to
interact with databases

SQL FACTS



SQL is a standardised language making it a very versatile skill used across different domains and tech environments



SQL FACTS



The modern data stack is moving to SQL-centric tools such as Snowflake, Amazon Redshift and Google BigQuery



SQL FACTS



SQL databases can store semi-structured data like JSON objects with expanding support for images, video and audio

SQL FACTS

SQL appears in over 40%
of all data job listings
posted on Indeed in 2021

SQL queries can be used
with many languages
and frameworks via
objects and APIs to
interact with databases

The modern data stack
is moving to SQL-centric
tools such as Snowflake,
Amazon Redshift and
Google BigQuery

65% of data analysts
and data scientists use
SQL according to a 2020
Stack Overflow survey

SQL is a standardised
language making it a
very versatile skill used
across different domains
and tech environments

SQL databases can store
semi-structured data
like JSON objects with
expanding support for
images, video and audio

THE PLAN FOR TODAY

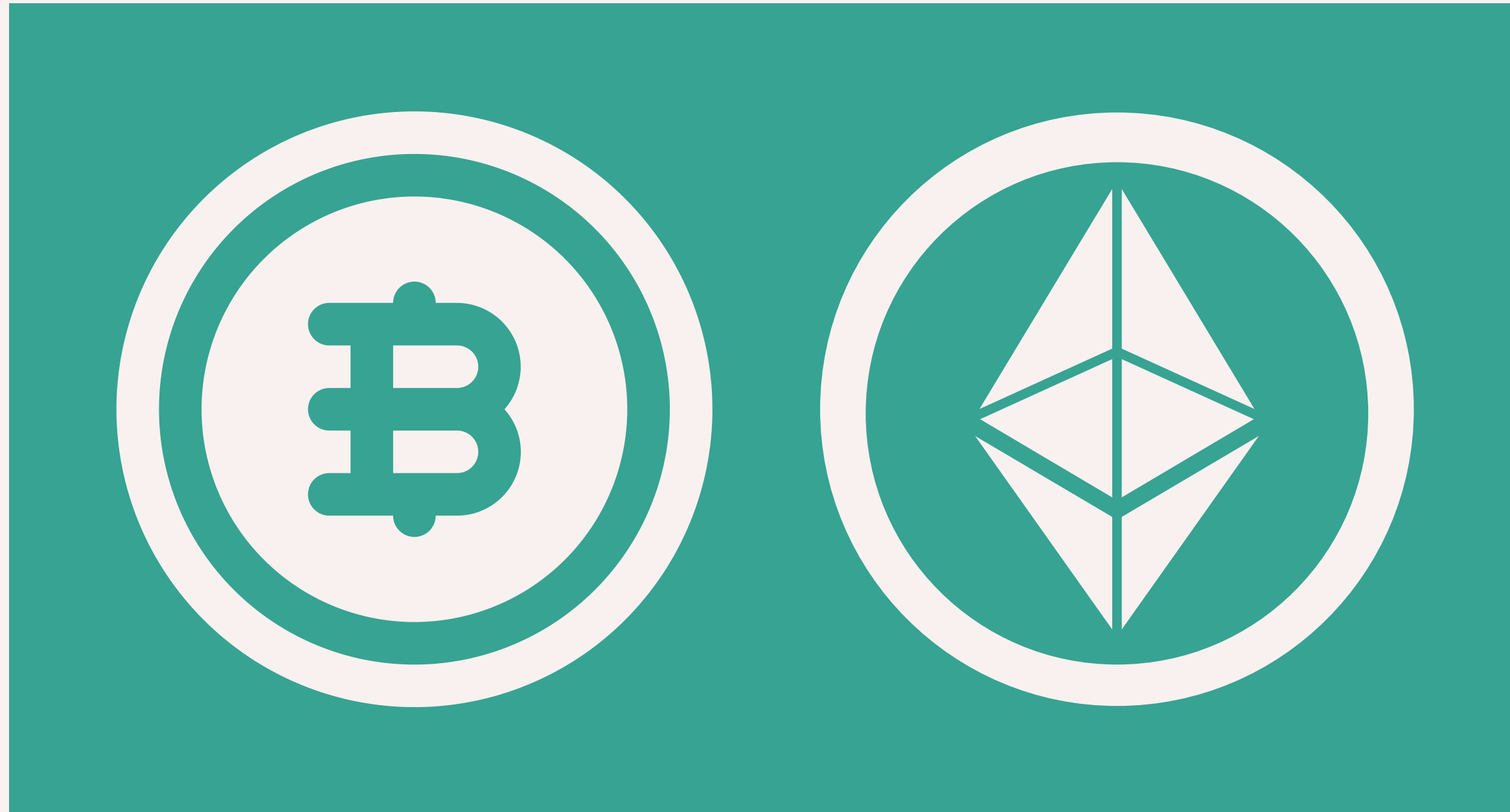


**SQL
Simplified**



**General
Q&A**

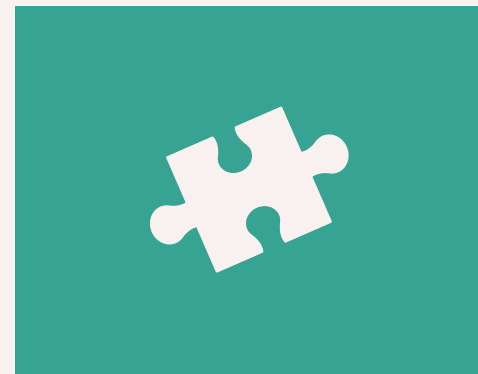
CRYPTO CASE STUDY



SQL SIMPLIFIED



Session 1
Case Study
Introduction



Session 2
Group By &
Case When



Session 3
Window
Functions



Session 4
Multiple
Table Joins

SQL SIMPLIFIED



Session 1

Case Study Introduction

SELECT STATEMENT



```
1 -- Return all columns and rows  
2 SELECT * FROM <table_name>;
```

LIMIT



```
1 -- Return the top 5 rows  
2 SELECT * FROM <table_name>  
3 LIMIT 5;
```

ORDER BY



```
1 -- Order in alphabetical order  
2 SELECT * FROM <table_name>  
3 ORDER BY <string_column>;
```

COUNT



```
1 -- Count all rows including nulls  
2 SELECT COUNT(*) FROM <table_name>;
```



```
1 -- Count non_null records in a column  
2 SELECT COUNT(<column>) FROM <table_name>;
```

WHERE FILTERS



```
1 -- Where filter on string column  
2 SELECT * FROM <table_name>  
3 WHERE <string_column> = '<exact_string>';
```



```
1 -- Where filter to exclude records  
2 SELECT * FROM <table_name>  
3 WHERE <string_column> != '<exact_string>';
```

DISTINCT



```
1 -- Get unique values from a table  
2 SELECT DISTINCT * FROM <table_name>;
```



```
1 -- Get unique values from a column  
2 SELECT DISTINCT <column> FROM <table_name>;
```

DISTINCT



```
1  -- Get unique values from 2+ columns
2  SELECT DISTINCT
3      <column_1>,
4      ...
5      <column_n>
6  FROM <table_name>;
```

SQL SIMPLIFIED



Session 1
Case Study
Introduction



Session 2
Group By &
Case When

BETWEEN



```
1 -- Where filter for date ranges  
2 SELECT * FROM <table_name>  
3 WHERE <date_column> BETWEEN '<start_date>' AND '<end_date>';
```

INEQUALITIES



```
1 -- Where filter using inequality
2 SELECT * FROM <table_name>
3 WHERE <column_name> {> or >= or < or <=} <exact_value>;
```

AGGREGATE FUNCTIONS



```
1 -- Min/max/average on a target column
2 -- using 1 single group by column
3 SELECT
4     <group_by_column>,
5     MIN(target_column),
6     MAX(target_column),
7     AVG(target_column)
8 FROM <table_name>
9 GROUP BY <group_by_column>;
```

EXTRACT DATE INFO



```
1 -- Extract a year date part from a date column
2 SELECT
3     EXTRACT(YEAR FROM <date_column>)
4 FROM <table_name>;
```

DATE TRUNCATION



```
1 -- Extract 1st day of month from date column
2 SELECT
3     DATE_TRUNC( 'MON', <date_column> )
4 FROM <table_name>;
```

CAST DATA TYPES



```
1 -- Cast float datatypes for rounding
2 SELECT
3     ROUND(AVG(<numeric_column>)::NUMERIC, 2)
4 FROM <table_name>;
```

MULTI-LEVEL SORTING



```
1 -- Multi-level sort with more than 1 column  
2 SELECT * FROM <table_name>  
3 ORDER BY <column_1> [desc], <column_2> [desc];
```

SQL SIMPLIFIED



Session 1

Case Study
Introduction



Session 2

Group By &
Case When

BASIC CASE WHEN



```
1 -- Basic case when statement structure
2 SELECT
3     CASE
4         WHEN <1st_if_condition> THEN <output_1>
5         WHEN <2nd_if_condition> THEN <output_2>
6         < Additional If-Then conditions go here >
7         ELSE <output_3>
8     END AS <alias_name>
```

SUM CASE WHEN: COUNTIF



```
1 -- Countif using sum case when
2 SELECT
3     SUM(
4         CASE
5             WHEN <if_condition> THEN 1
6             ELSE 0
7         END
8     )
9 FROM <table_name>;
```

SUM CASE WHEN: SUMIF

```
1 -- Sumif using sum case when
2 SELECT
3     SUM(
4         CASE
5             WHEN <if_condition> THEN <then_output>
6             ELSE 0
7         END
8     )
9 FROM <table_name>;
```

LEFT/RIGHT STRINGS



```
1 -- Find the n-th character from the left or right
2 SELECT
3     LEFT(<string_column>, n) AS left_nth,
4     RIGHT(<string_column>, n) AS right_nth
5 FROM <table_name>;
```

CHARACTER LENGTH



```
1 -- Find the character length of column  
2 SELECT LENGTH(<string_column>) FROM <table_name>;
```

REMOVE LAST CHARACTER

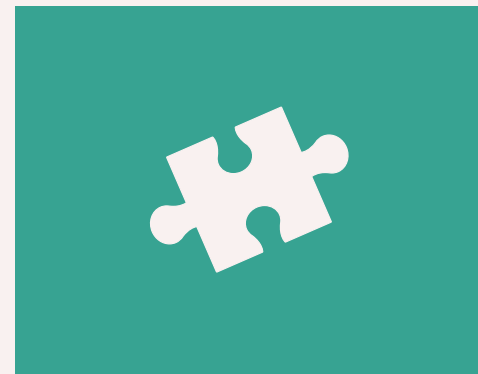


```
1 -- Remove the last character from string
2 SELECT
3     LEFT(<string_column>, LENGTH(<string_column>) - 1))
4 FROM <table_name>;
```

SQL SIMPLIFIED



Session 1
Case Study
Introduction



Session 2
Group By &
Case When



Session 3
Window
Functions

RANK WINDOW FUNCTION



```
1 -- Ranking window function
2 SELECT
3     RANK( ) OVER (
4         PARTITION BY <partition_column> ...
5         ORDER BY <sort_column> ...
6     ) AS ranking
7 FROM <table_name>;
```

7 DAY MOVING AVERAGE

```
1 -- 7 day moving average window function
2 SELECT
3     AVG(<target_column>) OVER (
4         PARTITION BY <partition_column> ...
5         ORDER BY <date_column> -- do not use DESC
6         RANGE BETWEEN '7 DAYS' PRECEDING AND CURRENT ROW
7     ) AS moving_avg
8 FROM <table_name>;
```

CUMULATIVE SUM



```
1 -- Cumulative sum window function
2 SELECT
3     SUM(<target_column>) OVER (
4         PARTITION BY <partition_column> ...
5         ORDER BY <date_column> -- do not use DESC
6         RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
7     ) AS cumulative_sum
8 FROM <table_name>;
```

LAG WINDOW FUNCTION

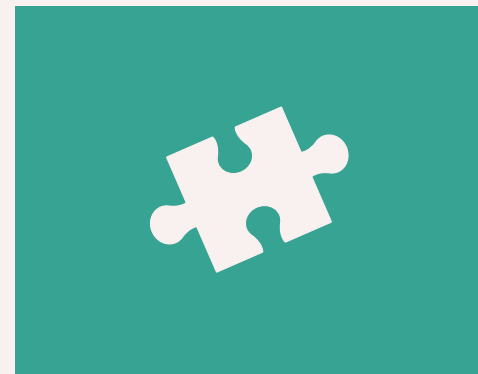


```
1 -- Lag window function
2 SELECT
3     LAG(<target_column>) OVER (
4         PARTITION BY <partition_column> ...
5         ORDER BY <date_column> -- do not use DESC
6     ) AS previus_target_value
7 FROM <table_name>;
```

SQL SIMPLIFIED



Session 1
Case Study
Introduction



Session 2
Group By &
Case When



Session 3
Window
Functions



Session 4
Multiple
Table Joins

INNER JOIN



```
1 -- Inner Join Between 2 Tables
2 SELECT
3     members.region,
4     SUM(transactions.quantity) AS bitcoin_quantity
5 FROM trading.transactions
6 INNER JOIN trading.members
7     ON transactions.member_id = members.member_id
8 WHERE transactions.ticker = 'BTC'
9     AND transactions.txn_type = 'BUY'
10 GROUP BY members.region
11 ORDER BY bitcoin_quantity DESC;
```

LEFT JOIN



```
1 -- Left Join Between 2 Tables
2 SELECT
3     prices.market_date,
4     COUNT(transactions.txn_id) AS transaction_count
5 FROM trading.prices
6 LEFT JOIN trading.transactions
7     ON prices.market_date = transactions.txn_date
8     AND prices.ticker = transactions.ticker
9 GROUP BY prices.market_date
10 HAVING COUNT(transactions.txn_id) < 5
11 ORDER BY prices.market_date DESC;
```

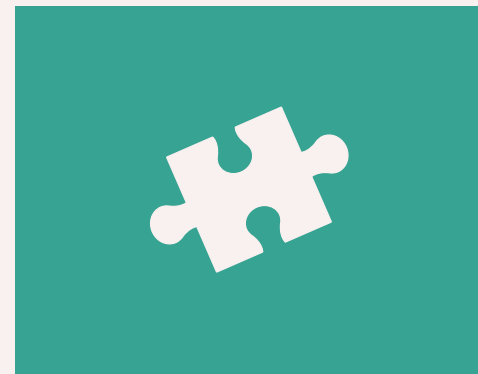
MULTIPLE TABLE JOINS

```
1 -- Multiple Table Joins
2 SELECT
3     members.region,
4     SUM(transactions.quantity) AS btc_quantity,
5     AVG(prices.price) AS avg_btc_price
6 FROM trading.transactions
7 INNER JOIN trading.prices
8     ON transactions.ticker = prices.ticker
9     AND transactions.txn_date = prices.market_date
10 INNER JOIN trading.members
11     ON transactions.member_id = members.member_id
12 WHERE transactions.ticker = 'BTC'
13     AND transactions.txn_type = 'BUY'
14 GROUP BY members.region
15 ORDER BY avg_btc_price DESC;
```

SQL SIMPLIFIED COMPLETED!



Session 1
Case Study
Introduction



Session 2
Group By &
Case When



Session 3
Window
Functions



Session 4
Multiple
Table Joins



General Q&A



bit.ly/dwd-info